ИТМО

Факультет Программной Инженерии и Компьютерной техники

РЕКОМЕНДАТЕЛЬНАЯ СИСТЕМА ВЫБОРА ТАНКА НА ОСНОВЕ БАЗЫ ЗНАНИЙ PROLOG

ЛАБОРАТОРНАЯ РАБОТА №2. СИСТЕМА ПОДДЕРЖКИ ПРИНЯТИЯ РЕШЕНИЙ

Аннотация – Разработана рекомендательная система (система поддержки принятия решений) для выбора боевого танка на основе базы знаний, представленной на языке Prolog. Пользователь вводит запрос на русском языке из трёх частей: что надо вернуть (список / рекомендация / что есть в наличии), объект запроса (танк, уровень, завод, нация) и набор фильтрующих условий (нация, тип техники, уровень, высокая альфа, максимальная скорость, минимальная перезарядка, конкретный танк). Условия накладываются на выборку в порядке их следования во входном строке. Программа на Python парсит строку запроса и посылает логические запросы к базе знаний на SWI-Prolog, которая выступает в роли источника данных (БД). На основе результатов выборки система формирует рекомендацию с обоснованием.

Ключевые слова: рекомендательная система, база знаний, Prolog, SWI-Prolog, системы поддержки принятия решений, танки

Выполнили

Ануфриев А.С., группа 465029

Проверил

Научный руководитель кафедры программной инженерии и вычислительной техники

27.10.2026

Введение

Целью данной работы является разработка программы — рекомендательной системы, которая использует базу знаний для предоставления рекомендаций на основе введённых пользователем данных. В качестве subject-домена выбраны боевые танки Второй мировой войны (база знаний из лабораторной работы №1, файл `tanks.pl`).

В отличие от первого варианта реализации (чистый Prolog-диалог), в этой работе архитектура разделена: Python-приложение отвечает за парсинг пользовательского ввода и формирование ответа, а база знаний на Prolog (`tanks.pl` вместе с уровнем доступа `tank_db.pl`) служит источником данных, к которому Python посылает логические запросы. Такой подход демонстрирует использование базы знаний как базы данных и логического вывода для фильтрации и ранжирования объектов.

Анализ требований

К системе предъявлены следующие требования:

- пользователь вводит запрос одной строкой на русском языке фиксированного формата из трёх частей;
- строка парсится на факты (режим вывода, объект запроса, условия);
- выполняются логические запросы к базе знаний на Prolog;
- система выдаёт рекомендацию (или список) с обоснованием.

Формат входного запроса

Запрос состоит из трёх частей в строгом порядке:

Часть	Назначение	Варианты
1	Что вернуть (режим)	`выведи` — список; `подскажи` — рекомендация с обосно
2	Объект	`танк` (по умолчанию), `уровень`, `завод`, `нация`
3	Условия (3.1–3.7)	нация, тип, уровень, высокая альфа, быстрый, скорос

Условия (часть 3) можно объединять; они накладываются на выборку в порядке их следования во входной строке:

№	Условие	Допустимые значения
3.1	Нация	`советский`, `немецкий`, `американский`
3.2	Тип техники	`тт` (тяжёлый), `ст` (средний), `лт` (лёгкий)
3.3	Уровень	`7лвл`, `8 лвл` (также `лвл 7/8`)
3.4	Высокая альфа	урон больше всех на уровне (по умолчанию 8 уровень)
3.5	Быстрый	скорость больше всех среди текущей выборки
3.6	Скорострельный	перезарядка меньше всех среди текущей выборки
3.7	Конкретный танк	отображаемое имя танка (например, `ИС-3`)

Режимы вывода:

- **выведи** — список всех объектов, прошедших фильтры, с характеристиками;
- **подскажи** — рекомендация одного объекта из прошедших фильтров с обоснованием (по каким условиям он подобран);
- **какой есть** — перечень доступных значений объекта (нации, уровни, заводы или имена танков) среди прошедших фильтров.

База знаний

База знаний `tanks.pl` содержит 18 танков 7 и 8 уровней трёх наций (СССР, Германия, США) трёх классов (тяжёлый, средний, ПТ-САУ). Каждый танк описан фактами:

- `tank/1` — идентификатор танка;
- `tank_stats/11` — уровень, нация, класс, HP, урон/выстрел (альфа), пробитие, броня, имя, калибр, перезарядка;
- `built_by/3` — производитель и год начала разработки;
- `unlocked_from/2` — базовый танк предыдущего уровня;
- `team/2`, `team_title/2` — комплекты по нациям и уровням.

Реализованы правила `durable/1` (живучесть, HP $\geq$ 1800), `hard_hitter/1` (пробитие $\geq$ 220 мм), `all_rounder/1`, `glass_cannon/1`, `opponents/2`, `best_gun/2`.

Для поддержки условий “быстрый” в уровень доступа `tank_db.pl` добавлены факты `speed/2` (ориентировочная максимальная скорость, км/ч) для всех 18 танков.

Архитектура и реализация

Компонент	Роль
`recommend.py`	Парсер русских запросов, формирование ответов, диалоговый
SWI-Prolog (`swipl`)	Механизм логического вызова, источник данных
`tank_db.pl`	Уровень доступа к данным поверх `tanks.pl`
`tanks.pl`	База знаний (факты и правила)

Python посылает запрос Prolog через стандартный ввод `swipl` (ключ `-t` в используемой сборке SWI-Prolog 10 некорректно обрабатывает цели с переменными, поэтому цель подаётся через stdin), парсит построчный вывод и формирует ответ на русском языке.

Основные предикаты `tank_db.pl`, вызываемые из Python:

Предикат	Назначение
`all_tanks(-Ts)`	все танки базы
`filter_tanks(+Nation,+Class,+Level,-Ts)`	фильтрация по нации/классу/
`alpha_max_tanks(+Level,-Ts)`	танки с максимальным уроно
`fastest_tanks(+Ids,-Ts)`	из списка — с максимальной
`fastest_reloading_tanks(+Ids,-Ts)`	из списка — с минимальной п
`tank_by_name(+Name,-Tank)`	поиск по отображаемому име
`tank_row(...)`	полная строка характеристик
`print_tank(+Tank)`	строка для парсинга Python,
`all_nations/1`, `all_levels/1`, `all_manufacturers/1`	справочники

Алгоритм обработки запроса:

1. Загружается вся таблица характеристиков из Prolog (`print_tank` для каждого танка).
2. Строка парсится слева направо: сначала режим вывода и объект, затем условия в порядке вхождения.
3. Начальная выборка — все танки; каждое условие — отдельный логический запрос к Prolog с пересечением с текущей выборкой.
4. Сверхъятивные условия (`альфа`, `быстрый`, `скорострельный`) применяются к текущей выборке: “урон больше всех на уровне” считается на указанном (или 8-м) уровне среди текущих кандидатов; “быстрый” и “скорострельный” — по текущей выборке.
5. Результат проецируется по объекту (танки / нации / уровни / заводы) и выводится согласно режиму.

Примеры запросов и результатов

Запрос	Режим	Результат
выведи танк нация советский лвл 8 с высокой альфой	список	ИСУ-152 максимум ветских 8
подскажи танк немецкий лвл 7 быстрый	рекомендация	Panther I рость 55 7-го уров
выведи танк американский скорострельный	список	T20 — пе мум сред ков
какой есть нации	наличие	Советски США
выведи танк конкретный танк ИС-3	список	ИС-3: HP битие 225
выведи танк тип лт	список	не найде танков)
подскажи танк лвл 8 с высокой альфой быстрый	рекомендация	ИСУ-152 (640) и л борке

Тестирование

Проведено 11 сквозных сценариев диалога (входной запрос → ответ системы). Все сценарии прошли: рекомендации соответствуют фактам из `tanks.pl`, пустые результаты обрабатываются сообщением о неудаче, команда выхода завершает программу с кодом 0.

Пример работы (`подскажи танк немецкий лвл 7 быстрый`):

```
Рекомендую
: Panther IОбоснование
: максимальнаяскорость  (55 кмч/).
Panther I Германия(, 7 лвл, средний): HP 1300, уронвыстрел/ 280,
   пробитие 198 мм,броня
 100 мм, калибр 88 мм, перезарядка 5.5 с, скорость 55 кмч/
```

Пример работы (`выведи танк тип ст лвл 8`):

```
Найдено
 3 танков():
1. T-44 Советский( Союз, 8 лвл, средний): ... скорость 53 кмч/
2. Panther II Германия(, 8 лвл, средний): ... перезарядка 5.14 с,
   скорость 46 кмч/
3. Pershing США(, 8 лвл, средний): ... скорость 40 кмч/
```

Заключение

Реализована рекомендательная система, в которой база знаний на Prolog используется как источник данных, а Python отвечает за парсинг естественноязычного (русского) запроса и представление результатов. Система корректно обрабатывает комбинируемые фильтры в порядке их следования,

три режима вывода и способы обоснования рекомендации. Возможные направления развития: расширение базы знаний новыми танками и характеристиками, добавление ранжирования по весу условий, графический интерфейс.

Приложение А. Листинг tank_db.pl

```
:- consult('tanks.pl').

speed(is_3, 37). speed(t_44, 53). speed(isu_152, 26).
speed(tiger_i, 45). speed(panther_i, 55). speed(jpanther, 46).
speed(tiger_ii, 34). speed(panther_ii, 46). speed(ferdinand, 22).
speed(t29, 35). speed(t_20, 40). speed(t_25_2, 40).
speed(t32, 35). speed(pershing, 40). speed(t28_prot, 13).
speed(is, 37). speed(t_43, 42). speed(su_152, 28).

filter_tanks(Nation, Class, Level, Ts) :-
    findall(T,
            (tank_stats(T, Lvl, Nat, Cls, _, _, _, _, _, _, _),
             (Nation = any; Nat = Nation),
             (Class = any; Cls = Class),
             (Level = any; Lvl = Level)),
            Ts).

alpha_max_tanks(Level, Ts) :-
    findall(Alpha-Tank,
            tank_stats(Tank, Level, _, _, _, Alpha, _, _, _, _, _),
            Pairs),
    (Pairs = [] -> Ts = [];
     keysort(Pairs, Sorted),
     last(Sorted, MaxAlpha-_),
     findall(Tank, (member(Alpha-Tank, Pairs), Alpha = MaxAlpha), Ts)).

fastest_tanks(Ids, Ts) :-
    findall(Speed-Tank, (member(Tank, Ids), speed(Tank, Speed)), Pairs),
    (Pairs = [] -> Ts = [];
     keysort(Pairs, Sorted),
     last(Sorted, MaxSpeed-_),
     findall(Tank, (member(Speed-Tank, Pairs), Speed = MaxSpeed), Ts)).
```

Приложение Б. Листинг recommend.py (фрагмент парсера)

```
def parse_request(text: str) -> dict:
    tokens = tokenize(text)
    mode, entity = None, None
    conditions = []  # (kind, value) впорядкевхождения
    i, n = 0, len(tokens)
    while i < n:
```

```
    t = tokens[i]
    if mode is None and t in MODE_ALIASES:
        mode = MODE_ALIASES[t]
    elif mode is None and t == "какой" and i + 1 < n and tokens[i +
       1] == "есть":
        mode = "what_exists"; i += 1
    elif entity is None and t in ENTITY_ALIASES:
        entity = ENTITY_ALIASES[t]
    elif t == "с" and i + 2 < n and tokens[i + 1] == "высокой" and
       tokens[i + 2] == "альфой":
        conditions.append(("alpha", None)); i += 2
    elif t in NATION_ALIASES:
        conditions.append(("nation", NATION_ALIASES[t]))
    ...
    i += 1
return {"mode": mode or "show", "entity": entity or "tank",
        "conditions": conditions}
```